

Local Machine Deployment Guide

Jima — CARAVAN Lounge

Version: 1.0	Updated: August 31, 2026
Prepared for: CARAVAN Lounge	Powered by: Idata Technologies
Deployment target: Local workstation/ server	Audience: Technical installer / system administrator

LOCAL USE ONLY — NOT A PRODUCTION DEPLOYMENT RECIPE

The audited backend currently runs TypeORM with `synchronize: true`, permits wildcard CORS, exposes Swagger and a seed operation, and performs required-account setup after startup. There is no verified migration baseline. Use only a new, disposable local PostgreSQL database. Never point this configuration at production, staging, shared, or valuable data. Production deployment is blocked until the controls in Documents 10 and 12 are completed.

Contents

1. Purpose, scope, audience	8. Schema and sample data
2. Audited technology stack	9. Verification checklist
3. Architecture and design	10. Operations, backup, restore, reset
4. Prerequisites	11. Troubleshooting
5. Repository layout	12. Security and acceptance
6. Development installation	13. Project references and change log
7. Production-style local mode	

1. Purpose, Scope, and Intended Audience

This guide enables an installer who receives the source repository to run the Jima restaurant management system on one local machine without any Replit knowledge. It covers source acquisition, workspace installation, a dedicated PostgreSQL database, environment variables, two-process startup, verification, backup/restore, and disposal.

In scope: Linux, macOS, and Windows through WSL2; local development mode; and a production-style build/start exercise on localhost. **Out of scope:** internet exposure, TLS, reverse proxies, cloud hosting, high availability, production hardening, migration creation, real account provisioning, and production release approval.

The intended reader can use a terminal, administer PostgreSQL, protect secrets, interpret HTTP responses, and stop services before database maintenance.

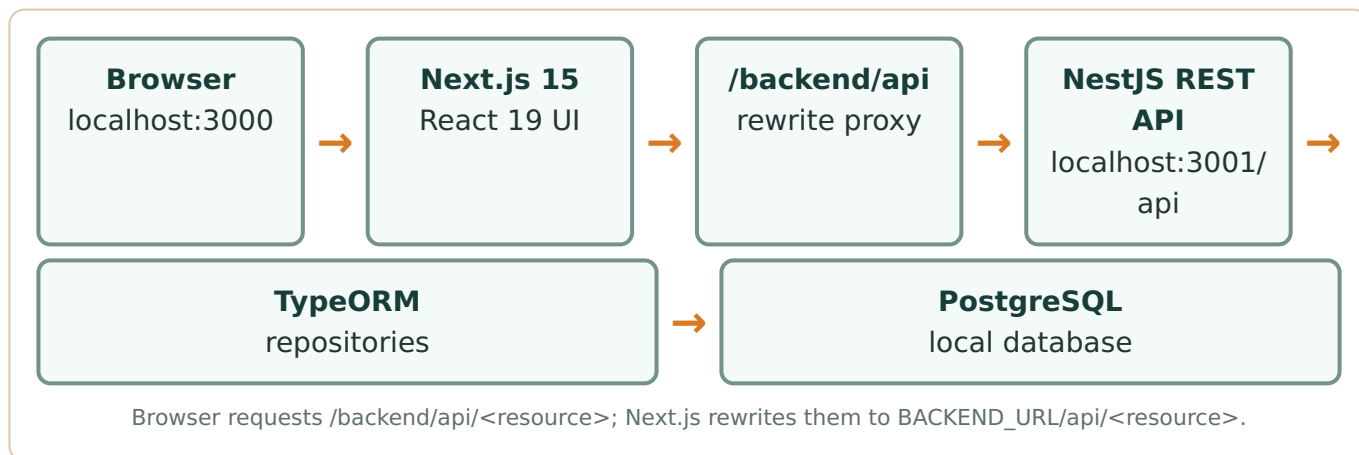
2. Audited Technology Stack

Versions below are package-file constraints from the audited repository. A caret means installation may resolve a compatible newer minor/patch; the workspace lockfile (`pnpm-lock.yaml` , format 9.0) supplies the reproducible resolution.

Layer	Technology / library	Repository version	Purpose
Runtime/ tooling	Node.js; TypeScript; pnpm	Node 20 LTS recommended; root TS <code>~5.9.3</code> ; lockfile 9.0	JavaScript runtime, typed source, monorepo package manager
Frontend	Next.js; React / React DOM	<code>^15.3.4</code> ; <code>19.1.0</code>	App Router web UI and same-origin API rewrite
Frontend libraries	Axios, Zustand, Tailwind CSS, Recharts, date-fns, ExcelJS, jsPDF, lucide-react	<code>^1.7.9</code> , <code>^5.0.3</code> , <code>^3.4.17</code> , <code>^2.15.3</code> , <code>^4.1.0</code> , <code>^4.4.0</code> , <code>^4.2.1</code> , <code>^0.469.0</code>	HTTP, auth state, styling, charts, dates, client exports, icons
Backend	NestJS core/common/ platform-express	<code>^10.4.15</code>	REST API modular monolith
Auth/API	Passport, passport-jwt, Nest JWT, Swagger	<code>^0.7.0</code> , <code>^4.0.1</code> , <code>^10.2.0</code> , <code>^7.4.2</code>	JWT authentication, RBAC guards, API explorer
Validation/ runtime	class-validator, class- transformer, RxJS, bcryptjs	<code>^0.14.1</code> , <code>^0.5.1</code> , <code>^7.8.1</code> , <code>^2.4.3</code>	DTO validation/ transformation, reactive primitives, password hashing
Persistence	PostgreSQL, TypeORM, pg driver, Nest TypeORM	Local PostgreSQL supported; <code>^0.3.21</code> , <code>^8.13.1</code> , <code>^10.0.2</code>	Relational storage and repository/data-source access

3. Architecture and Design

Jima is a **modular monolith**: one Next.js frontend process and one NestJS backend process, with backend business modules deployed together. PostgreSQL is the only authoritative persistence tier. The browser must not receive database or JWT secrets.



3.1 Layer responsibilities

Construct	Authoritative responsibility
Next.js pages/ components/store	Presentation, forms, role-aware navigation, loading/error states, client auth state, and Axios calls. Client checks are not security boundaries.
Controllers	REST routes, request/response boundary, decorators, guards, validated DTO hand-off.
DTOs and global validation pipe	Describe accepted payloads and transform/whitelist decorated fields. DTO coverage is currently incomplete, so services must remain authoritative.
JWT and role guards	Authenticate bearer JWTs and compare required roles. Passport JWT supplies the request user; role decorators drive RBAC.
Services	Business rules, workflow transitions, restaurant/branch scope checks, calculations, orchestration, and transaction boundaries.
TypeORM repositories / EntityManager	Scoped queries, persistence, pessimistic locks, and transaction participation.
Entities	Relational model, columns, associations, enums, and schema metadata currently consumed by synchronization.
PostgreSQL	Durable rows, relations, constraints, locking, commit, and rollback.

3.2 Authentication, authorization, and isolation

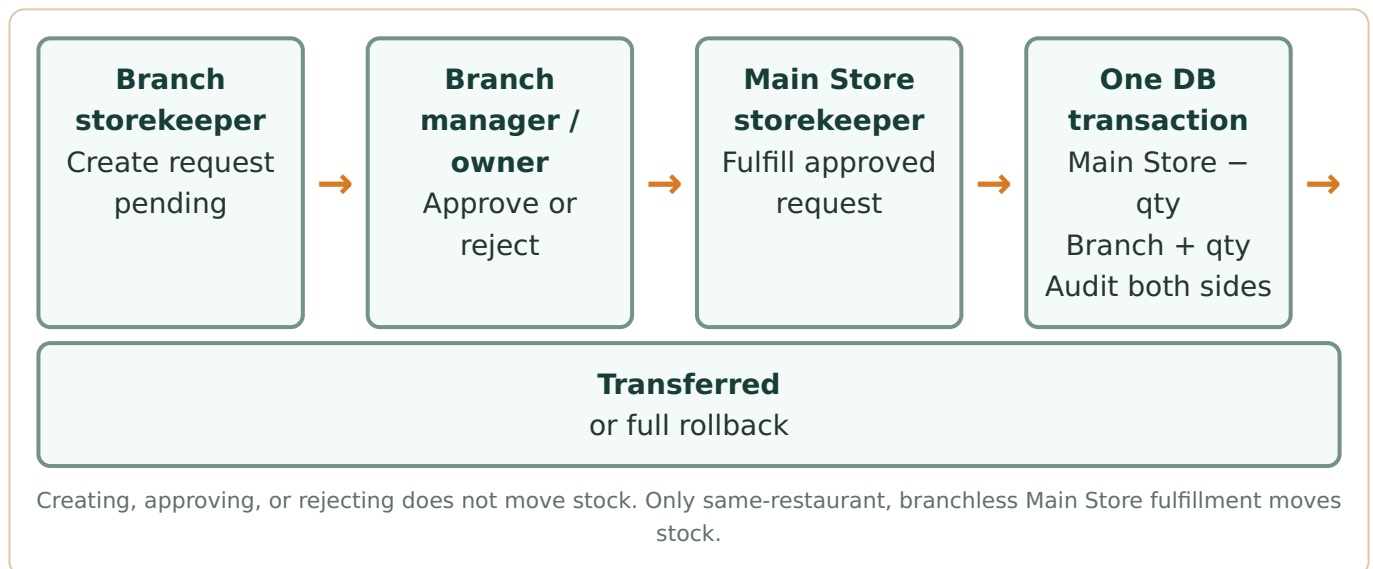
Login verifies a bcrypt hash and returns a stateless signed JWT. Axios attaches the bearer token; `JwtAuthGuard` validates it and `RolesGuard` enforces role metadata. Backend services—not hidden UI controls—must enforce all permissions. Records and mutations are restaurant-scoped; branch-assigned roles are additionally constrained to their assigned branch. Owners can operate at restaurant scope. A branchless Main Store storekeeper has the central-store fulfillment role.

3.3 Domains and transaction boundaries

Backend modules are Auth, Users, Restaurants, Branches, Menus, Tables, Orders, Kitchen, Payments, Summary, Inventory, Main Store, Notifications, and Seed. Important entities include users, branches, menu categories/items, tables, orders/items, payments/shifts, suppliers/purchase orders, branch inventory/adjustments/requests, and Main Store items/movements/receipts/transfers.

A transaction should encompass every all-or-nothing business change. The audited code uses explicit TypeORM transactions in Main Store, inventory, seed, and summary operations, with pessimistic locks in critical inventory flows. Main Store fulfillment is the clearest boundary: all requested lines, both stock sides, audit rows, balances, actor/time, and final status commit together or completely roll back. The audit notes that order, payment, table, and some other workflows still require atomicity review before production use.

3.4 Main Store / branch inventory workflow



4. Hardware and Software Prerequisites

Resource	Minimum	Recommended
CPU	2 x64/ARM64 cores	4+ modern cores
RAM	4 GB available	8 GB+ available
Free disk	5 GB plus database growth/backups	10 GB+ on SSD plus separate backup space
Network	Package/repository access during installation	Local firewall; no public port exposure

- **Node.js 20 LTS** and Corepack; use pnpm through Corepack.
- **PostgreSQL** server and CLI tools: `psql`, `createdb`, `dropdb`, `pg_dump`, `pg_restore`. PostgreSQL 15 or 16 is recommended locally.
- **Git** and a current Chrome, Edge, or Firefox browser.

- Ports **3000** (Next.js) and **3001** (NestJS) available on loopback.

```
# Verify on Linux, macOS, or WSL2
node --version
corepack --version
git --version
psql --version
pg_dump --version
```

4.1 Platform installation notes

Ubuntu/Debian Linux

```
sudo apt update
sudo apt install -y git postgresql postgresql-client
# Install Node.js 20 LTS using your organization's approved Node repository/version manager.
sudo systemctl enable --now postgresql
corepack enable
corepack prepare pnpm@9 --activate
pnpm --version
```

macOS (Homebrew)

```
brew install git node@20 postgresql@16
brew services start postgresql@16
export PATH="$(brew --prefix node@20)/bin:$(brew --prefix postgresql@16)/bin:$PATH"
corepack enable
corepack prepare pnpm@9 --activate
pnpm --version
```

Windows: use WSL2

WINDOWS REQUIREMENT

Use Ubuntu under WSL2 and keep the repository in the WSL filesystem (for example `~/src`), not under `/mnt/c`, for reliable permissions and performance. Run PostgreSQL inside WSL2 or supply a reachable dedicated local PostgreSQL server. Commands in this guide are bash/WSL commands, not PowerShell commands.

```
# Run in elevated PowerShell once, then reboot if requested:
wsl --install -d Ubuntu

# Then run inside the Ubuntu/WSL terminal:
sudo apt update
sudo apt install -y git postgresql postgresql-client
sudo service postgresql start
# Install Node.js 20 LTS in WSL using your approved method, then:
corepack enable
corepack prepare pnpm@9 --activate
```

5. Repository Layout and Correct Packages

Path/package	Use for this system?	Meaning
<code>artifacts/nextjs-app</code> <code>@workspace/nextjs-app</code>	Yes	Canonical restaurant frontend; run on port 3000.
<code>artifacts/nestjs-server</code> <code>@workspace/nestjs-server</code>	Yes	Authoritative restaurant REST API; run on port 3001.
<code>artifacts/api-server</code>	No	Separate minimal API artifact; it is not the restaurant API.
<code>artifacts/habesha</code>	No	Registered Vite/routing shim; do not run it as the local restaurant frontend.
<code>artifacts/mockup-sandbox</code>	No	Design/component preview only; not the restaurant application.
<code>docs</code> , <code>exports</code>	Reference	Project documentation and generated deliverables.

Always issue workspace commands from the repository root and select the two canonical packages by package name.

6. Step-by-Step Local Development Installation

Step 1 — Acquire and inspect the source

```
# Bash / macOS / WSL2
git clone "<repository-url>" "<project-directory>"
cd "<project-directory>"
git status --short
test -f package.json && test -f pnpm-lock.yaml
test -f artifacts/nextjs-app/package.json
test -f artifacts/nextjs-server/package.json
```

If source is delivered as an archive, extract it into `<project-directory>`, enter that directory, and perform the same file checks. Confirm the delivery source and checksum through a trusted channel.

Step 2 — Install locked dependencies

```
cd "<project-directory>"
corepack enable
corepack prepare pnpm@9 --activate
pnpm install --frozen-lockfile
```

The root preinstall rule rejects npm/yarn. Do not delete or regenerate `pnpm-lock.yaml` merely to make installation pass. The workspace also enforces a 1-day minimum package release age.

Step 3 — Create a disposable PostgreSQL role and database

Use the PostgreSQL administrative account appropriate to the OS. The following opens an SQL session:

```
# Ubuntu/WSL2
sudo -u postgres psql

# macOS Homebrew (current macOS user is commonly an administrator)
psql postgres
```

At the `postgres=#` SQL prompt, replace every angle-bracket placeholder. Keep the double quotes around identifiers and single quotes around the password:

```
-- SQL (psql), not bash
CREATE ROLE "<db-user>" LOGIN PASSWORD '<strong-local-password>';
CREATE DATABASE "<db-name>" OWNER "<db-user>" ENCODING 'UTF8';
\connect "<db-name>"
SELECT current_database(), current_user;
\quit
```

Use a unique local password and an empty database. Do not reuse production names or credentials. Commands with `DROP DATABASE` later in this guide are destructive.

Step 4 — Configure backend environment

Create `artifacts/nestjs-server/.env` locally. This is a template only; substitute locally generated values:

```
# artifacts/nestjs-server/.env
DATABASE_URL=postgresql://<db-user>:<url-encoded-strong-local-password>@localhost:5432/<db-name>
JWT_SECRET=<generated-random-secret>
PORT=3001
NODE_ENV=development
```

`DATABASE_URL` and `JWT_SECRET` are required. The code accepts `SESSION_SECRET` as a fallback if `JWT_SECRET` is absent, but relying on that fallback is discouraged: define a dedicated, long, random `JWT_SECRET`. Generate it locally without printing it into documentation or chat, and store it only in the untracked environment file.

Percent-encode reserved URL characters in the username/password portion of the connection URL: for example `@` becomes `%40`, `:` becomes `%3A`, `/` becomes `%2F`, `#` becomes `%23`, and `%` becomes `%25`. Do not add `sslmode=require` for the usual local non-TLS PostgreSQL service; the audited code enables SSL when that string is present.

Step 5 — Configure frontend environment

Create `artifacts/nextjs-app/.env.local`:

```
# artifacts/nextjs-app/.env.local
BACKEND_URL=http://localhost:3001
PORT=3000
```

`BACKEND_URL` is read by the Next.js server and targets NestJS. Never put `DATABASE_URL` or `JWT_SECRET` here, and never expose either through a `NEXT_PUBLIC_` variable. Never commit any `.env` or `.env.local` file. Check before committing:

```
git status --short
git check-ignore artifacts/nestjs-server/.env artifacts/nextjs-app/.env.local
```

Step 6 — Initialize the schema safely

No migration baseline currently exists. On first backend startup, TypeORM `synchronize: true` compares registered entities and automatically creates/updates schema objects. There is no separate migration command to run. Treat startup itself as schema initialization, and only allow it against the disposable database created above.

Step 7 — Start backend (Terminal 1)

```
cd "<project-directory>"
pnpm --filter @workspace/nestjs-server run dev
```

Expected listener: `http://localhost:3001`. Startup also invokes an audited required-account setup routine after listening; it may create or update restaurant, branch, and staff records. Therefore even “unseeded” startup mutates the local database. This is another reason to use only disposable data.

Step 8 — Start frontend (Terminal 2)

```
cd "<project-directory>"
pnpm --filter @workspace/nextjs-app run dev
```

Windows/native-shell warning: the exact frontend development script begins with `rm -rf .next`. Use WSL2 or Git Bash. Native PowerShell/cmd users must manually remove `artifacts/nextjs-app/.next` and then invoke an equivalent Next.js command; WSL2 is the supported path in this guide.

Open `http://localhost:3000/login`. The browser API boundary is `http://localhost:3000/backend/api/*`; do not change browser calls to `/api`.

Step 9 — Optional sample-data seed (disposable local DB only)

The protected `POST /api/seed` endpoint requires an authenticated ADMIN bearer token and writes demonstration restaurant, branch, menu, table, inventory, supplier, and user data. It also contains repository-defined demonstration account credentials; this guide intentionally does not publish any account passwords.

```
# Only after authorized local ADMIN credentials are supplied securely:
curl -X POST "http://localhost:3001/api/seed" \
  -H "Authorization: Bearer <local-admin-jwt>" \
  -H "Content-Type: application/json"
```

Do not seed production, staging, shared, or valuable data. Use a newly created disposable database, take a snapshot first if needed, and inspect the response. The operation's "already seeded" check is limited; do not assume it is a general merge/import tool.

7. Production-Style Local Build and Start

This mode validates compiled artifacts locally; it does not make the audited application production-ready. Keep the same disposable database and secrets. Stop all development processes first.

Never run `next build` while `next dev` is active. Both use `artifacts/nextjs-app/.next` and can race, corrupt output, or produce a blank/broken preview.

7.1 Build with exact package scripts

```
cd "<project-directory>"

# Backend package script: "build": "nest build"
pnpm --filter @workspace/nestjs-server run build

# Frontend package script: "build": "next build"
pnpm --filter @workspace/nextjs-app run build
```

7.2 Start in two terminals

```
# Terminal 1 – backend package script: "start": "node dist/main"
cd "<project-directory>"
NODE_ENV=production pnpm --filter @workspace/nestjs-server run start

# Terminal 2 – frontend package script: "start": "next start -p ${PORT:-3000}"
cd "<project-directory>"
NODE_ENV=production pnpm --filter @workspace/nextjs-app run start
```

The package-local environment files still provide database/JWT, backend target, and ports. URLs remain frontend `http://localhost:3000`, backend `http://localhost:3001`, and Swagger `http://localhost:3001/api/docs`.

8. Database Schema and Seed Behavior

Behavior	Operator implication
<code>synchronize: true</code>	Backend startup automatically creates or changes local tables to match entities. It is convenient for disposable development but unsafe as a production change process.
No verified migration baseline	There is no trustworthy migration history for controlled promotion/rollback. Do not improvise a production migration from this guide.
Startup account setup	After the listener starts, the service ensures required restaurant/branch/staff records and may update selected existing accounts. It is not a read-only startup.
Protected seed endpoint	ADMIN JWT required. Adds demonstration domain data and is intended only for a disposable local environment.
Seed credentials	Repository code defines demonstration passwords. Do not publish them, reuse them, or treat them as production provisioning.

For a clean schema, create a new empty database and start the backend once. For a repeatable reset, stop both processes, back up anything intentionally retained, drop/recreate the disposable database, and restart. Never connect this backend configuration to production.

9. Verification Checklist

9.1 Connectivity and routing

- ☐ `curl -i http://localhost:3001/` returns HTTP 200 and `{"status":"ok"}`.
- ☐ `curl -i http://localhost:3001/nestjs-backend` returns HTTP 200 and the same status.
- ☐ `http://localhost:3001/api/docs` renders Swagger and lists the Jima Restaurant API.
- ☐ `http://localhost:3000/login` renders the login UI.

- ❑ An invalid login fails explicitly (normally 401) and does not navigate to a protected page or issue a usable token.
- ❑ After authorized local login, `GET http://localhost:3000/backend/api/auth/me` with the bearer token returns only the authenticated user permitted by the API.

```
# Authenticated same-origin proxy check (token placeholder only)
curl -i "http://localhost:3000/backend/api/auth/me" \
-H "Authorization: Bearer <local-jwt>"
```

9.2 Session and browser behavior

- ❑ Refresh a protected page: the authorized session restores correctly, or redirects safely if expired.
- ❑ Direct navigation to a protected URL while logged out redirects/denies access.
- ❑ Logout clears client auth state; refreshing or using Back does not restore protected data.
- ❑ Clear stale local browser storage after changing `JWT_SECRET`, then sign in again.

9.3 Workflow and isolation smoke tests

- ❑ Role-specific navigation and backend denials agree for admin, owner, manager, coordinator, waiter, chef, cashier, and storekeeper.
- ❑ Create an order, coordinate/assign it, progress kitchen states, serve, pay, and confirm the table is released.
- ❑ Exercise inventory receipt/adjustment/request flows and verify balances and audit records.
- ❑ Branch storekeeper submits a Main Store request for only the assigned branch; request/approval/rejection alone changes no stock.
- ❑ Assigned manager or restaurant owner decides; another branch/restaurant and disallowed roles are denied.
- ❑ Only the same-restaurant branchless Main Store storekeeper fulfills an approved request.
- ❑ Successful multi-line fulfillment atomically decrements Main Store, increments only the destination branch, writes both audit sides, and marks transferred.
- ❑ Insufficient stock on any line leaves every line, audit record, destination item, and status unchanged.

10. Startup, Shutdown, Restart, Logs, and Data Operations

10.1 Normal operations

Action	Procedure
Start	Start PostgreSQL; start backend in Terminal 1; wait for listening/database messages; start frontend in Terminal 2; run health checks.

Shutdown	Press <code>Ctrl+C</code> once in frontend, then backend. Confirm ports are released. Stop PostgreSQL only if no other local databases need it.
Restart	Stop the affected process cleanly, correct configuration/code, then rerun its package command. Restart both after changing <code>JWT_SECRET</code> or <code>BACKEND_URL</code> .
Logs	Development/build/start output is written to each terminal. Capture it using an approved local log mechanism if needed; redact tokens, headers, connection URLs, passwords, and stack traces containing sensitive paths.

```
# Identify listeners if a clean shutdown failed
lsof -nP -iTCP:3000 -sTCP:LISTEN
lsof -nP -iTCP:3001 -sTCP:LISTEN
# Linux/WSL alternative
ss -ltnp | grep -E ':3000|:3001'
```

10.2 Backup with `pg_dump`

Stop application writes for a simple consistent local maintenance point, create a protected directory, and use the custom archive format:

```
mkdir -p "<backup-directory>"
pg_dump --format=custom --no-owner --no-acl \
  --dbname="postgresql://<db-user>:<url-encoded-strong-local-password>@localhost:5432/<db-name>" \
  --file="<backup-directory>/jima-local.backup"
test -s "<backup-directory>/jima-local.backup"
pg_restore --list "<backup-directory>/jima-local.backup" | head
```

Command-line URLs may be visible in shell history/process lists. Prefer a protected `.pgpass`, an interactive prompt, or your organization's credential method. Never commit backups; they can contain personal and operational data.

10.3 Restore to a newly created database

```
# Bash / macOS / WSL2 – application stopped
createdb --host=localhost --username="<db-user>" "<restore-db-name>"
pg_restore --host=localhost --username="<db-user>" \
  --dbname="<restore-db-name>" --no-owner --no-acl \
  "<backup-directory>/jima-local.backup"
```

Point a temporary backend environment at `<restore-db-name>` only after verifying the target name, then perform health and record-count checks. Because synchronization runs on connection, preserve an untouched copy of the archive before testing.

Plain SQL alternative

```
# Create a plain backup
pg_dump --format=plain --no-owner --no-acl \
  --dbname="postgresql://<db-user>:<url-encoded-strong-local-password>@localhost:5432/<db-name>" \
  --file="<backup-directory>/jima-local.sql"

# Restore plain SQL into an empty database
createdb --host=localhost --username="<db-user>" "<restore-db-name>"
psql --host=localhost --username="<db-user>" \
  --dbname="<restore-db-name>" --file="<backup-directory>/jima-local.sql"
```

10.4 Cleanup/reset of disposable local data

Destructive: stop backend/frontend and verify the exact database name. Never substitute a production/shared name.

```
# Bash / macOS / WSL2
dropdb --host=localhost --username="<db-user>" --if-exists "<db-name>"
createdb --host=localhost --username="<db-user>" --owner="<db-user>" "<db-name>"
# Restart backend once to recreate the synchronized schema.
```

Final removal, after confirming no database remains owned by the role:

```
# SQL in an administrative psql session
DROP ROLE "<db-user>";
```

11. Troubleshooting

Symptom	Likely cause	Safe resolution
Missing <code>DATABASE_URL</code> ; TypeORM cannot connect	Backend <code>.env</code> absent, wrong directory, or malformed URL	Place it in <code>artifacts/nestjs-server/.env</code> ; verify placeholders were replaced and password is URL-encoded. Do not paste the value into logs/tickets.
<code>JWT_SECRET</code> or <code>SESSION_SECRET</code> is required	No signing secret loaded	Define a long random <code>JWT_SECRET</code> in backend <code>.env</code> . Do not rely on the fallback and do not expose it to Next.js/browser.
Frontend proxy returns <code>ECONNREFUSED</code> / 502	Backend stopped, wrong <code>BACKEND_URL</code> , wrong port	Check backend health directly on 3001, set <code>BACKEND_URL=http://localhost:3001</code> , restart Next.js.
404 from <code>/api</code> in browser	Wrong request boundary	Browser calls must use <code>/backend/api</code> . Direct backend calls use <code>http://localhost:3001/api</code> . Do not run the separate api-server artifact.
<code>EADDRINUSE</code> on 3000/3001	Old process or another service owns port	Inspect with <code>lsof / ss</code> , stop the owner cleanly, or coordinate both <code>PORT</code> and <code>BACKEND_URL</code> changes.
PostgreSQL password/authentication failure	Role/password/host mismatch; local auth policy	Test with <code>psql</code> , inspect local <code>pg_hba.conf</code> , reset only the dedicated local role password, then encode it in the URL.
PostgreSQL SSL error	Local server is non-TLS but URL includes <code>sslmode=require</code> , or TLS expectations differ	Remove <code>sslmode=require</code> for normal local PostgreSQL. The code turns on SSL when that exact option appears.
<code>rm</code> not recognized on Windows	Native cmd/PowerShell ran frontend dev script	Use WSL2/Git Bash. Otherwise manually delete <code>.next</code> before an equivalent command. WSL2 is recommended.
401 after secret/user change	Stale JWT in Zustand/localStorage	Logout; clear site data/localStorage for localhost; sign in again using securely supplied local credentials.
Missing chunks, build errors, blank page	<code>next build</code> and <code>next dev</code> shared <code>.next</code>	Stop all Next.js processes, delete <code>artifacts/nextjs-app/.next</code> , then run exactly one mode.
TypeORM schema/permission errors	Database not empty, wrong owner, insufficient create/alter rights, or incompatible old synchronized schema	Confirm this is disposable; back it up; recreate an empty database owned by the local role. Never “fix” by granting broad production rights.

Blank preview / wrong UI	Wrong artifact started, frontend backend unavailable, stale assets, browser error	Run only the named Next.js/NestJS packages, health-check backend, stop Next.js, remove <code>.next</code> , restart, and hard refresh.
-----------------------------	---	---

12. Security and Operational Checklist

- ☐ Database is new, disposable, locally scoped, and contains no production/valuable data.
- ☐ PostgreSQL, 3000, and 3001 are bound/firewalled for local access only; no router port forwarding.
- ☐ Unique strong database password and long random JWT secret are stored only in ignored local files/approved secret storage.
- ☐ No secrets, database URLs, tokens, account passwords, or backups appear in Git, chat, screenshots, terminal captures, or tickets.
- ☐ Frontend has no database/JWT variables and no secret uses a `NEXT_PUBLIC` prefix.
- ☐ Only `@workspace/nextjs-app` and `@workspace/nestjs-server` are running.
- ☐ Sample seeding is restricted to disposable local data and uses an authorized ADMIN token supplied securely.
- ☐ Swagger and wildcard CORS are understood as local-only behavior, not accepted production controls.
- ☐ Dependencies came from the locked pnpm install; the lockfile and release-age safeguard were not bypassed.
- ☐ Backups are access-controlled, nonempty, restorable, retained only as required, and securely deleted afterward.
- ☐ Logs are reviewed without leaking secrets; processes are stopped cleanly before restore/reset.
- ☐ No production approval is inferred from a successful production-style local build.

13. Local Deployment Acceptance Checklist

- ☐ Prerequisite versions and available resources are recorded.
- ☐ Source provenance is trusted; correct repository and package names are confirmed.
- ☐ Locked install completes with pnpm under Node 20 LTS.
- ☐ Dedicated database/role and both local environment files are configured without committed secrets.
- ☐ Backend health routes, Swagger, frontend login, and same-origin proxy work.
- ☐ Invalid login, authenticated `auth/me`, refresh, protected navigation, and logout behave safely.
- ☐ Role and restaurant/branch isolation checks pass.

- Order/payment/table and Main Store/branch inventory smoke tests pass, including atomic failure.
- Development mode and, separately, production-style build/start complete without a `.next` race.
- A local backup is nonempty and a restore into a different empty database has been verified.
- Shutdown/restart and disposable reset procedures are understood and documented locally.
- Installer records exceptions; unresolved auth, isolation, stock, payment, schema, or restore failures reject acceptance.

14. Project References

The repository documents below are part of the audited context. Relative links work when this HTML remains in `docs/`; Markdown files may require a local editor/viewer.

1. 01 — Functional Requirements Specification
2. 02 — Software Requirements Specification
3. 03 — Software Design Document
4. 04 — User Manual
5. 05 — Handoff and Deployment Requirements
6. 06 — Deploying on Vercel
7. 07 — Architecture and Routing
8. 08 — Environment and Secrets
9. 09 — API and Authorization Boundary
10. 10 — Database Safety Runbook
11. 11 — Test and Release Runbook
12. 12 — Production Readiness Status
13. 13 — Comprehensive System Test Plan

15. Change Log

Version	Date	Change	Prepared for
1.0	August 31, 2026	Initial audited local-machine deployment guide, including development and production-style local modes, database safety, verification, operations, and troubleshooting.	CARAVAN Lounge, powered by Idata Technologies

End of document — Jima — CARAVAN Lounge Local Machine Deployment Guide v1.0.